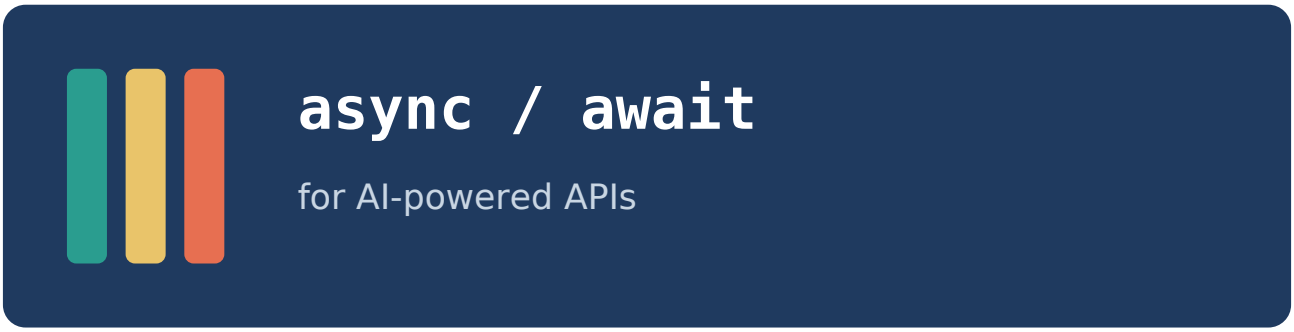


Building Async AI APIs with FastAPI

A short, practical guide to the async concept: what it is, why AI APIs need it, and how to use it correctly in FastAPI, with tested Python examples.

Created: Tuesday, 27 October 2026, 9:24 PM (PDT)
Level: Beginner to intermediate Python
Tested with: Python 3.13, FastAPI 0.162, HTTPX 0.28

Index

1. Why Async Matters for AI APIs 3

2. Sync vs Async: The Core Idea 3

2.1 The restaurant analogy 3

2.2 See it in code 4

3. The Async Building Blocks 4

3.1 Coroutines: functions that can pause 4

3.2 The event loop 5

3.3 Tasks: running things side by side 5

4. Async in FastAPI 6

4.1 The golden rule, measured 6

5. Building an Async AI API 8

5.1 Setup: one shared async HTTP client 8

5.2 A basic async endpoint with a timeout 8

5.3 Fan-out: many prompts at once 9

5.4 Streaming tokens to the user 9

5.5 Blocking SDKs and CPU work: move them to a thread 10

6. Run and Test It 10

7. Common Mistakes Checklist 11

8. Summary 12

References 12

1. Why Async Matters for AI APIs

When your API calls an AI model (OpenAI, Anthropic, a local model server, a vector database), your code spends almost all of its time **waiting**. A typical LLM call takes 1 to 30 seconds, and during that time your server's CPU is doing nothing useful for that request.

This kind of work is called **I/O-bound**: the slow part is input/output (network, disk, database), not computation. Async programming is a way to use that waiting time to serve other users.

Work type	What is slow	AI API example	Best tool
I/O-bound	Waiting on network / disk	Calling an LLM API, querying a vector DB, fetching a URL	**async / await**
CPU-bound	Heavy computation	Running a model locally, tokenizing huge text, image resizing	Threads / processes / workers

> **Key idea**
> Async does not make a single AI call faster. It lets one server handle **many** AI calls at the same time, because it never sits idle while waiting.

2. Sync vs Async: The Core Idea

2.1 The restaurant analogy

A **synchronous** waiter takes your order, walks to the kitchen and stands there until your food is ready. Only then does he serve the next table. An **asynchronous** waiter gives your order to the kitchen, then immediately takes the next table's order. When any dish is ready, he delivers it. Same waiter, same kitchen, far more tables served.

In this picture the waiter is your server's thread, the kitchen is the AI model, and the tables are users.

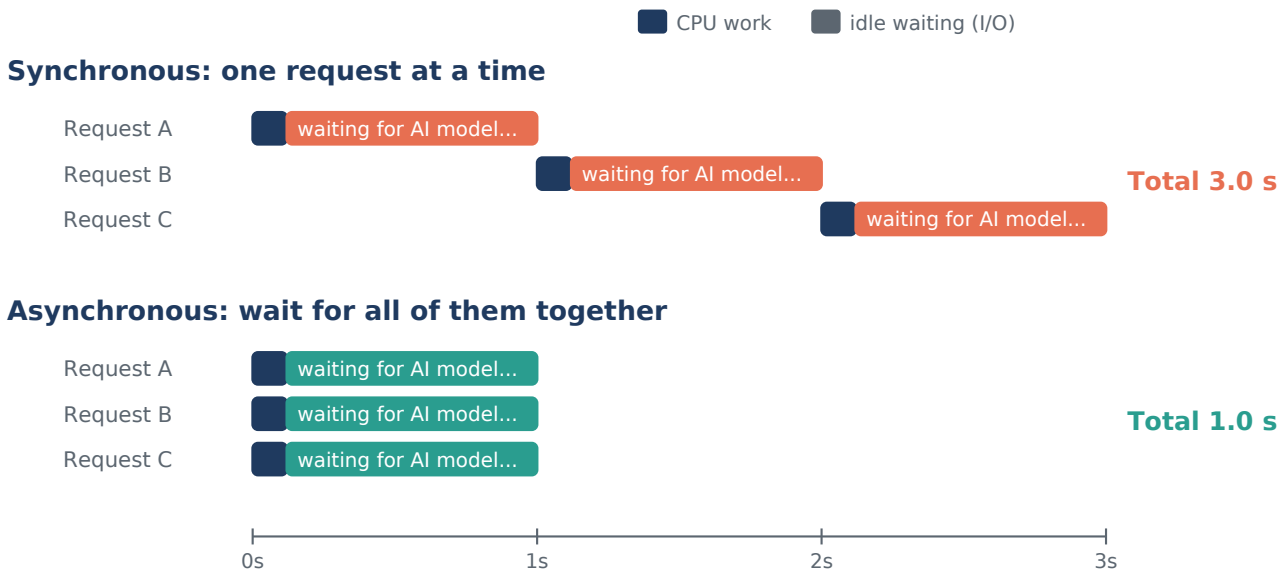


Figure 1. Three AI calls that each take 1 second. Sync: 3 s in total. Async: about 1 s, because the waits overlap.

2.2 See it in code

Below, `time.sleep(1)` and `asyncio.sleep(1)` stand in for a 1-second AI model call. The only difference between the two versions is how they wait.

sync_vs_async.py

```python
import asyncio, time

def call_model_sync(prompt):
    time.sleep(1)              # pretend: waiting for the AI model
    return f"answer to {prompt}"

async def call_model_async(prompt):
    await asyncio.sleep(1)     # waiting, but lets others run
    return f"answer to {prompt}"

def run_sync():
    start = time.perf_counter()
    results = [call_model_sync(p) for p in ["A", "B", "C"]]
    print(f"sync : {time.perf_counter() - start:.1f}s", results)

async def run_async():
    start = time.perf_counter()
    results = await asyncio.gather(
        *(call_model_async(p) for p in ["A", "B", "C"]))
    print(f"async: {time.perf_counter() - start:.1f}s", results)

run_sync()                 # sync : 3.0s
asyncio.run(run_async())   # async: 1.0s
```

Output

```
sync : 3.0s ['answer to A', 'answer to B', 'answer to C']
async: 1.0s ['answer to A', 'answer to B', 'answer to C']
```

3. The Async Building Blocks

You only need four ideas to understand async Python:

Concept	What it is	In one line
`async def`	Defines a **coroutine** function	A function that is allowed to pause
`await`	A **pause point**	"I am waiting; let someone else run"
Event loop	The **scheduler**	Runs one coroutine at a time, switching at every await
Task	A coroutine **scheduled** on the loop	Runs in the background while you do other things

3.1 Coroutines: functions that can pause

Calling an `async def` function does **not** run it. It returns a coroutine object, like a recipe that has not been cooked yet. Something must run it: either `await` (inside another async function) or `asyncio.run()` (at the top of a script).

coroutine.py

```python
import asyncio

async def greet():
    return "hello"

coro = greet()          # nothing runs yet!
print(type(coro))       # <class 'coroutine'>
print(asyncio.run(coro))  # hello  (the event loop runs it)
```

3.2 The event loop

The event loop is the heart of async. It runs on a **single thread** and executes one task at a time. When a task reaches `await` on something slow (like an HTTP call to an LLM), the loop parks it and switches to another ready task. When the slow operation finishes, the parked task becomes ready again and continues from where it stopped.

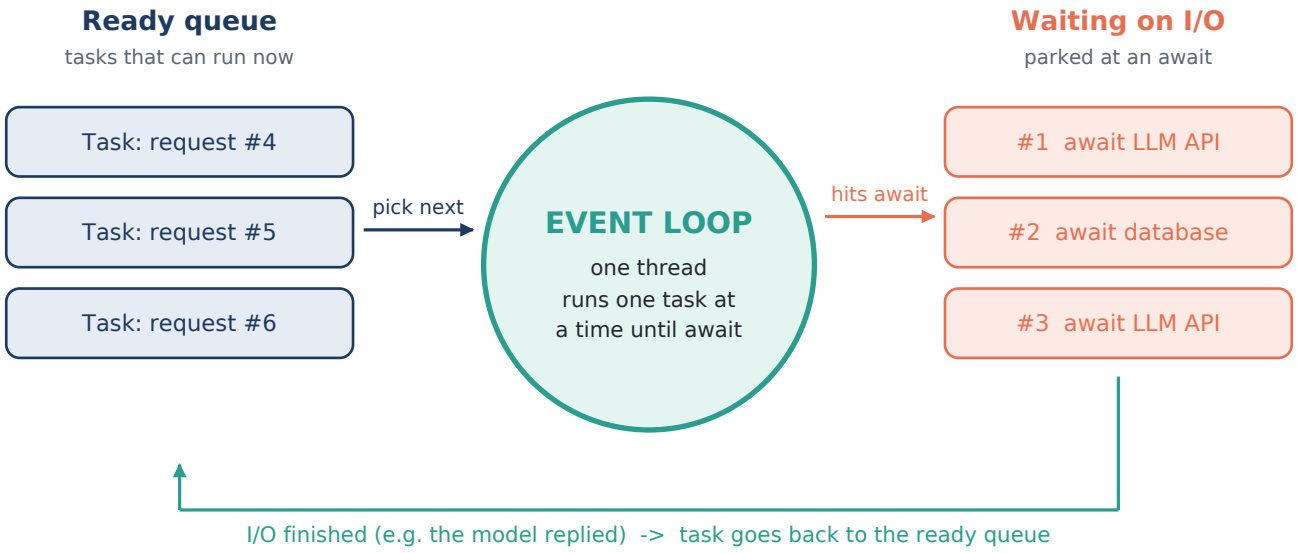


Figure 2. The event loop switches between tasks at every await. No task is ever 'waiting' while holding the thread.

Important consequence

Switching only happens at an **await**. If a task runs slow code *without* awaiting (for example `time.sleep()`, `requests.get()` or a heavy loop), the whole loop is frozen and every other user waits. This is called **blocking the event loop**.

3.3 Tasks: running things side by side

`asyncio.create_task()` schedules a coroutine to start right away in the background. `asyncio.gather()` is a shortcut that runs many coroutines together and returns all results in order.

tasks.py

```python
import asyncio

async def step(name, seconds):
    print(f"{name} started")
    await asyncio.sleep(seconds)   # pause point: loop switches here
    print(f"{name} finished")
    return name

async def main():
    t1 = asyncio.create_task(step("embed", 2))
    t2 = asyncio.create_task(step("search", 1))
    print(await t1, await t2)

asyncio.run(main())
```

Output

```
embed started
search started
search finished
embed finished
embed search
```

Notice that "search" finished first even though it started second: both were waiting at the same time. Total time is 2 s, not 3 s.

4. Async in FastAPI

FastAPI runs on an ASGI server (usually Uvicorn), which owns an event loop. FastAPI looks at **how you define** each endpoint and decides where to run it:

- `async def` endpoint: runs directly **on the event loop**. Fast and light, but you must only use non-blocking (awaitable) code inside it.
- `def` endpoint: FastAPI runs it in a **thread pool**, so blocking code is safe there. Threads cost more memory, so this scales less well than async.

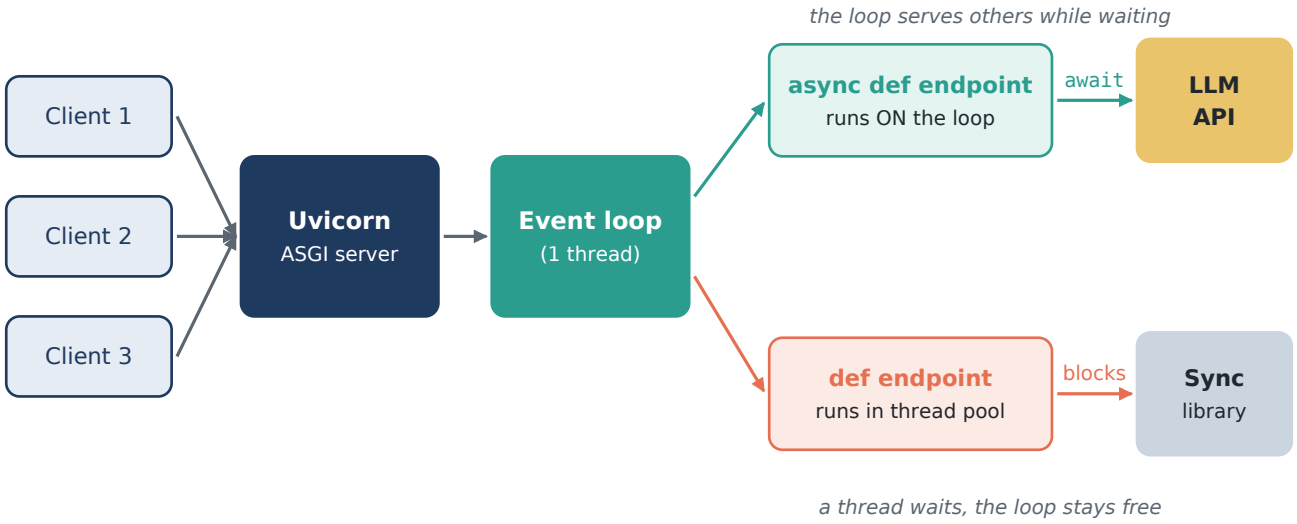


Figure 3. How a request travels through FastAPI. async def uses the loop; def is moved to a worker thread.

4.1 The golden rule, measured

Three endpoints that each wait 1 second, but in different ways:

blocking_demo.py

```python
import asyncio, time
from fastapi import FastAPI

app = FastAPI()

@app.get("/good-async")
async def good_async():
    await asyncio.sleep(1)    # non-blocking wait
    return {"ok": True}

@app.get("/bad-async")
async def bad_async():
    time.sleep(1)             # BLOCKS the whole event loop!
    return {"ok": True}

@app.get("/plain-def")
def plain_def():
    time.sleep(1)             # OK: FastAPI runs def in a thread pool
    return {"ok": True}
```

Result of sending **5 requests at the same time** to each endpoint (measured):

Endpoint	Total time	Why
`/good-async`	**1.0 s**	All 5 waits overlap on the loop
`/bad-async`	**5.0 s**	Each request freezes the loop; they run one by one
`/plain-def`	**1.0 s**	Each request gets its own thread

The golden rule

Use `async def` only when everything slow inside it is **awaited** (async libraries like `httpx.AsyncClient`, the official async AI SDKs, `asyncpg`). If you must call a blocking library, use plain `def`, or push the call to a thread with `asyncio.to_thread()` (see 5.5). The worst choice is `async def` with blocking code inside.

5. Building an Async AI API

Now we combine everything into a small but realistic AI API. It calls an LLM over HTTP and shows the five patterns you will use most. The full file is **app.py**; it is shown in parts below. The LLM URL is a placeholder: point it at your provider, or use the provider's async SDK (for example `AsyncOpenAI` or `AsyncAnthropic`), which follows the same await pattern.

5.1 Setup: one shared async HTTP client

Creating a new HTTP client per request is slow. With FastAPI's `lifespan` we create **one** `httpx.AsyncClient` at startup, reuse its connections for every request, and close it on shutdown. The `Semaphore` limits how many model calls run at once so we do not hit the provider's rate limits.

app.py (part 1): setup

```python
import asyncio
import os
from contextlib import asynccontextmanager

import httpx
from fastapi import FastAPI, HTTPException
from fastapi.responses import StreamingResponse
from pydantic import BaseModel

LLM_URL = os.getenv("LLM_URL", "https://api.example.com/v1/generate")
MAX_PARALLEL = 5                          # max calls to the model at once
limiter = asyncio.Semaphore(MAX_PARALLEL)

@asynccontextmanager
async def lifespan(app: FastAPI):
    # One shared client = reused connections (fast). Created once at startup.
    app.state.http = httpx.AsyncClient(timeout=30.0)
    yield
    await app.state.http.aclose()         # clean shutdown

app = FastAPI(lifespan=lifespan)

class Prompt(BaseModel):
    text: str

class BatchPrompt(BaseModel):
    texts: list[str]

async def ask_llm(text: str) -> str:
    async with limiter:                   # wait here if 5 calls are running
        r = await app.state.http.post(LLM_URL, json={"prompt": text})
        r.raise_for_status()
        return r.json()["output"]
```

5.2 A basic async endpoint with a timeout

While `await ask_llm(...)` waits for the model, the event loop serves other users. `asyncio.timeout()` (Python 3.11+) cancels the call if the model is too slow, so a stuck request cannot hang forever.

app.py (part 2): /generate

```python
@app.post("/generate")
async def generate(p: Prompt):
    try:
        async with asyncio.timeout(20):  # Python 3.11+
            answer = await ask_llm(p.text)
    except TimeoutError:
        raise HTTPException(504, "Model took too long")
    return {"answer": answer}
```

5.3 Fan-out: many prompts at once

`asyncio.gather()` sends all prompts together. The semaphore from part 1 keeps at most 5 in flight; the rest wait for a free slot. In our test, 10 prompts with a 1-second model took **2.0 s** instead of 10 s.

app.py (part 3): /batch

```python
@app.post("/batch")
async def batch(b: BatchPrompt):
    answers = await asyncio.gather(*(ask_llm(t) for t in b.texts))
    return {"answers": answers}
```

asyncio.gather() of 10 prompts with Semaphore(5)

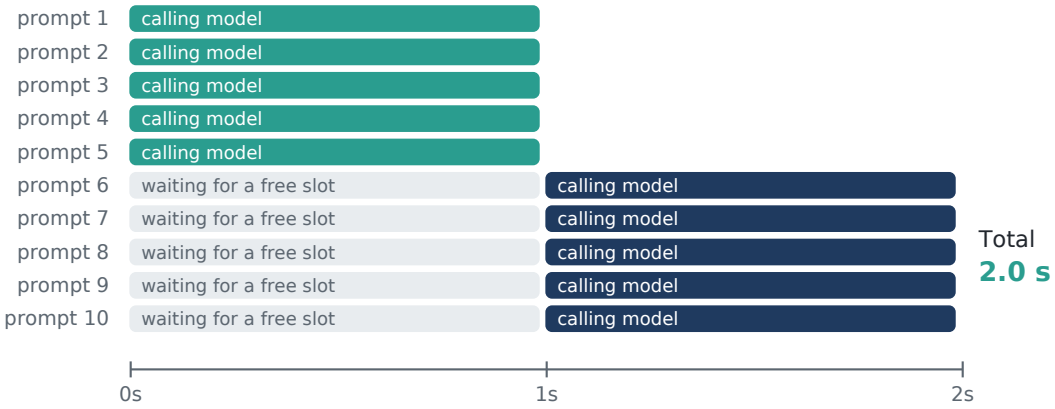


Figure 4. The semaphore turns 10 parallel calls into two waves of 5: fast, but polite to the AI provider.

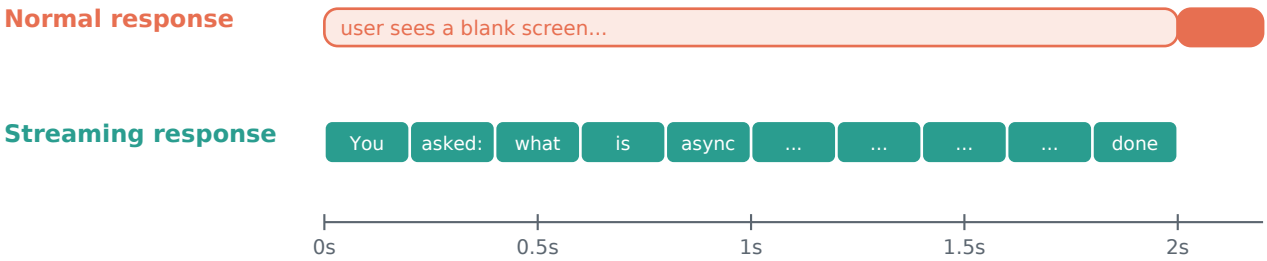
5.4 Streaming tokens to the user

LLMs produce text word by word. Instead of making the user wait for the whole answer, return a `StreamingResponse` fed by an **async generator** (an `async def` that uses `yield`). Each `await` inside the generator frees the loop between tokens. The `data: ...` lines follow the Server-Sent Events (SSE) format that browsers understand.

app.py (part 4): /stream

```python
async def fake_token_stream(text: str):
    for word in f"You asked: {text}".split():
        await asyncio.sleep(0.2)          # model "thinking" per token
        yield f"data: {word}\n\n"         # Server-Sent Events format

@app.post("/stream")
async def stream(p: Prompt):
    return StreamingResponse(fake_token_stream(p.text),
                             media_type="text/event-stream")
```



First word arrives at 0.2 s instead of 2 s. Same total time, much better experience.

Figure 5. Streaming does not shorten the total time, but the user starts reading almost immediately.

5.5 Blocking SDKs and CPU work: move them to a thread

Some libraries are sync-only (an older SDK, a local model, PDF parsing). Calling them directly in an `async def` would block the loop. `asyncio.to_thread()` runs the function in a worker thread and lets you **await** the result. In our test, 3 requests that each block for 1 s finished in **1.0 s** total.

app.py (part 5): /sentiment

```
def blocking_sentiment(text: str) -> str:
    import time; time.sleep(1)            # e.g. old sync SDK / local model
    return "positive" if "good" in text else "neutral"

@app.post("/sentiment")
async def sentiment(p: Prompt):
    label = await asyncio.to_thread(blocking_sentiment, p.text)
    return {"label": label}
```

> **Heavy CPU work**
> Threads are fine for blocking I/O. For truly heavy computation (e.g. running a large model on CPU), Python threads still share one interpreter lock, so prefer a separate worker process, a task queue (Celery, ARQ) or a dedicated model server, and call it from FastAPI with an async HTTP client.

6. Run and Test It

Install the packages and start the server:

Terminal

```
pip install fastapi uvicorn httpx
export LLM_URL="https://your-llm-provider/v1/generate"
uvicorn app:app --reload
```

Open **http://127.0.0.1:8000/docs** for the interactive Swagger UI, or call the API from Python. The client below is itself async, so it can fire several requests at once, which is a nice way to watch the server handle them concurrently:

client.py

```
import asyncio, httpx

async def main():
    async with httpx.AsyncClient(base_url="http://127.0.0.1:8000",
                                 timeout=60) as client:
        r = await client.post("/batch",
                              json={"texts": ["What is async?",
                                              "What is an event loop?"]})
        print(r.json())

        # read the stream token by token
        async with client.stream("POST", "/stream",
                                 json={"text": "hello"}) as resp:
            async for line in resp.aiter_lines():
                if line:
                    print(line)

asyncio.run(main())
```

To test without a real LLM, we replaced the HTTP client with `httpx.MockTransport` returning a fake answer after 1 second. These are the measured results quoted in this guide:

Output

```
/generate          1 prompt            -> 1.0 s
/batch             10 prompts, limit 5 -> 2.0 s
/stream            tokens every 0.2 s  -> 'data: You', 'data: asked:', ...
/sentiment         3 blocking calls    -> 1.0 s  (via asyncio.to_thread)
```

7. Common Mistakes Checklist

Mistake	Why it hurts	Fix
`time.sleep()` or `requests.get()` inside `async def`	Freezes the event loop for all users	`await asyncio.sleep()`, `httpx.AsyncClient`
Forgetting `await`	You get a coroutine object, not the result (and a warning)	Always `await` coroutine calls
New HTTP client per request	Slow: new connection + TLS every time	One shared client in `lifespan`
Unlimited parallel AI calls	Rate-limit errors (429) from the provider	`asyncio.Semaphore`
No timeout on model calls	Stuck requests pile up forever	`asyncio.timeout()` / client timeout
Sync AI SDK in async code	Blocks the loop on every call	Use the async SDK, or `asyncio.to_thread()`
Heavy CPU work in the API	Slows every request	Worker process, task queue or model server

8. Summary

- AI APIs are mostly **waiting** (I/O-bound). Async lets one server use that waiting time to serve other requests.
- `async def` creates a coroutine; `await` is where it may pause; the **event loop** switches tasks at every await.
- Never block the loop. Async libraries inside async endpoints; blocking code in `def` endpoints or `asyncio.to_thread()`.
- Run things concurrently with `asyncio.gather()`, protect providers with a `Semaphore`, and protect users with timeouts.
- Share one `httpx.AsyncClient` via `lifespan`, and stream long answers with `StreamingResponse`.

References

1. FastAPI: Concurrency and async / await
   https://fastapi.tiangolo.com/async/
2. FastAPI: Lifespan Events
   https://fastapi.tiangolo.com/advanced/events/
3. FastAPI: Custom Response (StreamingResponse)
   https://fastapi.tiangolo.com/advanced/custom-response/
4. Python docs: asyncio (Asynchronous I/O)
   https://docs.python.org/3/library/asyncio.html
5. Python docs: Coroutines and Tasks (gather, timeout, to_thread)
   https://docs.python.org/3/library/asyncio-task.html
6. Python docs: Synchronization primitives (Semaphore)
   https://docs.python.org/3/library/asyncio-sync.html
7. HTTPX: Async Support
   https://www.python-httpx.org/async/
8. Uvicorn: ASGI web server
   https://www.uvicorn.org/
9. MDN: Server-sent events
   https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events